

SHOPMIND

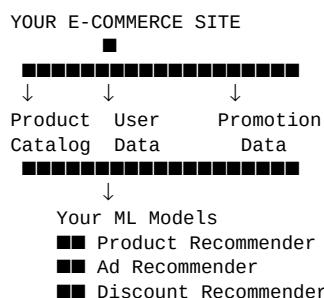
Dataset & Product Catalogue Architecture

Building a realistic e-commerce recommendation, advertising, and discount recommendation platform

A practical architecture and dataset plan for training the models, serving recommendations, and presenting the system through an interactive e-commerce sandbox.

1. Objective

The ShopMind project is designed as an end-to-end e-commerce recommendation sandbox. The website should behave like a realistic online shopping platform while the underlying ML systems independently handle product recommendation, advertisement recommendation and ranking, and discount / coupon recommendation.

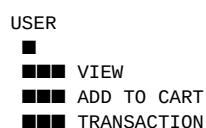


2. Recommended Dataset Stack

Purpose	Recommended dataset
Product recommendation	RetailRocket Recommender System Dataset
Large product catalogue	Amazon Reviews 2023
Coupon recommendation	Ponpare Coupon Purchase Prediction
Discount / redemption modeling	Alibaba / Tianchi O2O Coupon Dataset
Advertisement CTR modeling	Criteo Display Advertising Dataset
Advertisement ranking	Criteo + simulated campaigns

3. Product Recommendation — RetailRocket

RetailRocket is a strong starting point because it contains real e-commerce behavioral events such as product views, add-to-cart events, transactions, timestamps, item properties, and categories.

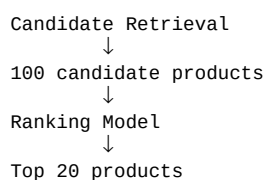


Training signal

A simple implicit-feedback weighting scheme can be: view → weak positive, add-to-cart → stronger positive, transaction → strongest positive.

```
interaction =
  1 → view
  3 → add-to-cart
  5 → transaction
```

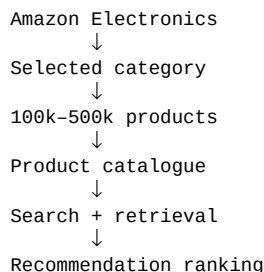
Possible implementations include item-item recommendation, matrix factorization, LightGCN, SASRec, two-tower retrieval, and learning-to-rank.



4. Large Product Catalogue — Amazon Reviews 2023

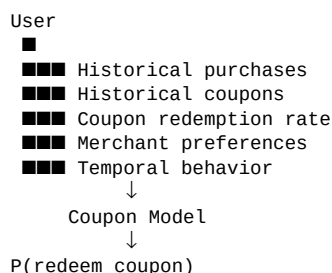
The Amazon Reviews 2023 dataset provides a very large product ecosystem and rich metadata, including product descriptions, prices, categories, images, brands, reviews, timestamps, and related-product information.

The full dataset is extremely large. A practical approach is to select one or two categories, such as Electronics, and build a manageable catalogue. The Amazon dataset should primarily serve as a catalogue/metadata source while RetailRocket can provide the main behavioral recommendation-training signal.



5. Discount Recommendation — Alibaba / Tianchi O2O Coupon

The discount model needs user, coupon, discount, time, behavior, and redemption information. The Tianchi / Alibaba O2O Coupon dataset contains user, merchant, coupon, discount, distance, date, and behavioral fields.



Because discount_rate is explicitly represented, the dataset can be used to study how discount levels relate to redemption behavior. It also contains multiple discount structures, including percentage discounts and spend-X-save-Y offers.

6. Coupon Recommendation — Ponpare

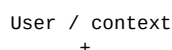
The Ponpare Coupon Purchase Prediction dataset contains approximately one year of transactional data for 22,873 users. It is directly related to recommending a ranked list of coupons to users.

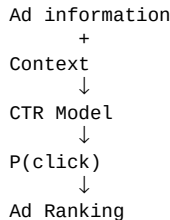
Useful fields include users, coupon metadata, coupon views, purchases, timestamps, sessions, and coupon areas.

Dataset	Best suited to
Tianchi O2O	Discount → redemption behavior
Ponpare	User → coupon recommendation

7. Advertisement Recommendation — Criteo

Criteo provides a standard benchmark for CTR prediction. Each impression contains a clicked/not-clicked label, 13 numerical features, and 26 categorical features. The complete 1TB dataset is unnecessary for this project; start with the smaller Display Advertising Challenge dataset.





Recommended progression: Logistic Regression → XGBoost / LightGBM → DeepFM → Two-tower retrieval + DeepFM ranking.

8. Product Catalogue Architecture

Option A — Static catalogue

For the serious version, download and preprocess a catalogue once and store it in PostgreSQL.

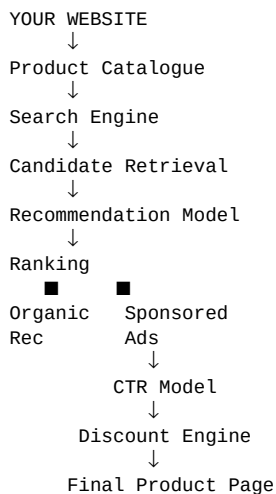
```
products
product_id
title
description
brand
category
subcategory
price
image
rating
stock
```

Use OpenSearch or Elasticsearch for product search. The website can expose hundreds of thousands of products without loading the entire catalogue into the browser.

Option B — Live product API

For an initial frontend prototype, a live product API such as DummyJSON can provide product cards, product pages, search, pagination, category filtering, sorting, and related metadata. Its small sample catalogue should be treated as a frontend/demo source rather than as the core training dataset.

9. Recommended Serving Architecture



10. Independent Model Training

Product recommendation

```
RetailRocket
↓
LightGCN
↓
Product embeddings
```

Advertisement model

```
Criteo
↓
DeepFM
↓
CTR prediction
```

Discount model

```
Tianchi
↓
XGBoost / Neural Model
↓
Coupon redemption probability
```

11. End-to-End Serving Flow

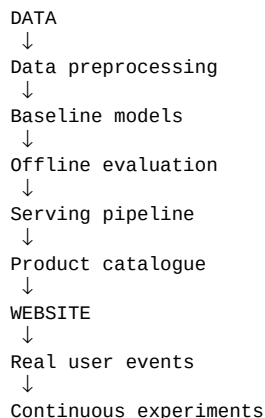
```
USER
■
■■■ Search / Click
■
↓
YOUR SYSTEM
■
■■■ Recommender
■■■ Ad Ranking
■■■ Discount Engine
■
↓
WEBSITE
■
↓
User feedback
■
↓
Event database
```

12. Optional Dataset — Microsoft MIND

MIND is a useful additional benchmark for impression-based ranking. It contains approximately 160k articles, 15M+ impression logs, and 1M users, with clicked and non-clicked impressions. It is not appropriate as the product catalogue for the Amazon-like site, but it is useful for studying impression-based ranking.

```
User history
+
Displayed candidates
+
clicked / not clicked
```

13. Project Development Order



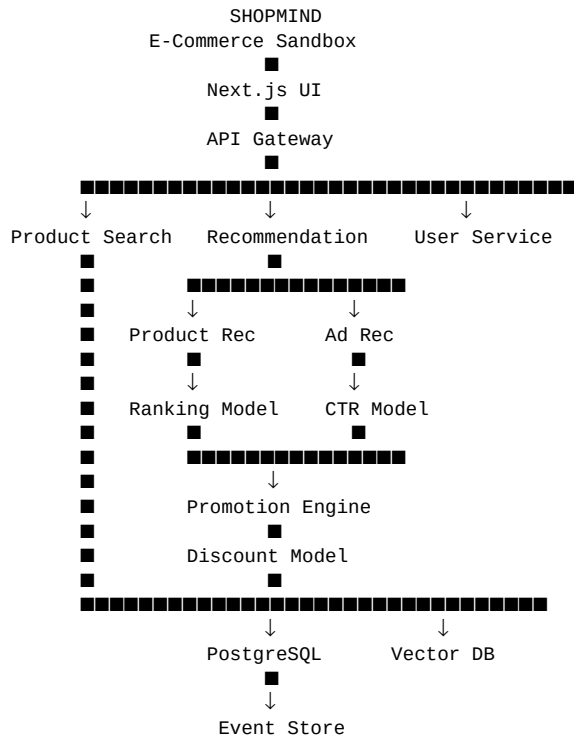
Start with the data and baseline models before building the full website. This ensures that recommendations, advertisements, and discounts are produced by genuine trained models rather than hardcoded UI behavior.

14. Suggested Technology Stack

Layer	Technology
Frontend	Next.js + TypeScript + Tailwind
Backend	FastAPI
Database	PostgreSQL
Search	OpenSearch / Elasticsearch
Vector search	pgvector
ML	PyTorch + LightGBM + scikit-learn
Experiment tracking	MLflow
Cache	Redis
Deployment	Docker
Frontend hosting	Vercel
Backend / cloud	Render / Railway / AWS

Not every component is required on day one. Start with PostgreSQL + FastAPI + one model + a simple frontend, then introduce the search, vector, caching, and experiment-tracking components as the system grows.

15. Final Architecture



16. Core Principle

The final system should maintain a clear separation between training data, ML models, model serving, product catalogue, website behavior, and newly collected user events.



The goal is for a visitor to interact with a realistic e-commerce site while the underlying system genuinely runs the recommendation, advertisement, and discount models. This turns the project from a collection of ML notebooks into an end-to-end recommender-system research and product platform.